# Viva Preparation - Member 1 - Auth Service

## Owned Component

Owned microservice:

- `auth-service`

Main files to show:

- `services/auth-service/src/app.js`
- `services/auth-service/src/controllers/authController.js`
- `services/auth-service/src/middleware/auth.js`
- `services/auth-service/src/models/User.js`
- `services/auth-service/Dockerfile`
- `sonar/auth-service.properties`

## What To Say

I implemented the authentication and identity service for EventHub. This service handles user registration, login, JWT generation, current-user lookup, role-based identity, and user lookup endpoints used by other services.

The service supports three main roles:

- `customer`
- `organizer`
- `admin`

The frontend does not call the service directly. It calls the public API gateway, and the gateway forwards authentication requests to `auth-service`.

## Endpoints To Explain

- `POST /api/auth/register` - creates a new user account.
- `POST /api/auth/login` - validates credentials and returns a JWT token.
- `GET /api/auth/me` - returns the currently logged-in user using the JWT token.
- `GET /api/auth/users/:id` - returns user details by ID.
- `GET /api/auth/users?role=admin` - supports internal lookup of admin users.

Through the gateway these become:

- `/auth/register`
- `/auth/login`
- `/auth/me`
- `/auth/users/:id`

## Integration Point

Integration to demonstrate:

`booking-service` calls `auth-service` to find admin users when a booking request is created. This allows the system to notify admins about booking activity.

Example explanation:

When a customer creates a booking, the booking service needs to notify admins. It calls the auth service to get admin users, then sends notification data to the notification service. This proves that the auth service is not isolated; another group member's service depends on it.

## Demo Steps

1. Open the deployed frontend.
2. Register or log in as a user.
3. Show that the login returns a token and stores the logged-in user.
4. Show role-based navigation after login.
5. Explain that protected requests include the JWT token.
6. During booking flow, explain that booking-service uses auth-service to find admin users.

## Security Points

- Passwords are not returned to the frontend.
- JWT is used for stateless authentication.
- Protected routes require `Authorization: Bearer <token>`.
- Role information is included in the authenticated user object.
- Secrets such as `JWT_SECRET` and MongoDB URI are configured through environment variables.
- In Azure, services are not exposed directly; the public access point is the API gateway.

## DevOps And Cloud Points

- The service has its own Dockerfile.
- The Docker image is published to GitHub Container Registry.
- The service is deployed to Azure Container Apps.
- The service has a separate SonarCloud project:
  - `EventHub Auth Service`
- The GitHub Actions workflow scans this service using:
  - `sonar/auth-service.properties`

## Likely Viva Questions

### What is the purpose of the auth service?

It manages user identity for the system. It handles registration, login, JWT token generation, current user retrieval, and user lookup for other services.

### Why did you use JWT?

JWT allows stateless authentication. After login, the client sends the token with each protected request, and services can verify the user without storing server-side sessions.

### How does another service use your service?

The booking service calls the auth service to fetch admin users. This is used when sending admin notifications for booking requests.

### How is your service secured?

It uses JWT authentication, protected middleware, role-based user information, environment variables for secrets, and it is deployed behind the gateway rather than being directly exposed publicly.

### What happens after login?

The user enters email and password. The auth service validates credentials, generates a JWT token, and returns the token plus user details. The frontend stores that data and uses it for future requests.

### What would happen if the JWT is missing or invalid?

Protected endpoints reject the request with an unauthorized response.

### Why does each microservice have its own database?

It keeps services loosely coupled and independently deployable. The auth service owns user data, while other services only request user information through APIs.

## Common Group Questions

### What is the application?

EventHub is a microservice-based event booking platform. Customers can browse events and request bookings, organizers can manage events and approvals, and admins can oversee activity.

### What are the four microservices?

- Auth Service
- Event Service
- Booking Service
- Notification Service

### What is the API gateway used for?

The gateway is the public backend entry point. The frontend calls the gateway, and the gateway routes requests to the correct internal service.

### What is the main integration flow?

The strongest flow is booking:

1. Customer requests a booking.
2. Booking service calls event service to check and reserve seats.
3. Booking service stores a pending booking.
4. Booking service calls notification service to create alerts.

5. Booking service calls auth service when admin user data is needed.

### What DevOps practices are implemented?

- GitHub repository
- GitHub Actions CI
- Docker containerization
- GHCR container image publishing
- Azure Container Apps deployment
- SonarCloud DevSecOps scanning

### What security practices are implemented?

- JWT authentication
- Role-based access
- Internal microservice ingress in Azure
- Environment variables and GitHub secrets
- SonarCloud static analysis
- Express Helmet middleware

## Quick Emergency Checks

Gateway health:

```
curl https://gateway.mangocoast-efcefaea.southeastasia.azurecontainerapps.io
```

Azure services:

```
az containerapp list --query "[].{name:name, resourceGroup:resourceGroup,
fqdn:properties.configuration.ingress.fqdn, status:properties.runningStatus}" -o
table
```

Auth logs:

```
az containerapp logs show --name auth-service --resource-group eventhub-rg --tail
100
```